Parte 1.b

Artigas: Rocha (400).
Canelones: Artigas (500), Colonia (200), Durazno (170), Punta del Este (90).
Colonia: Montevideo (180).
Durazno: no hay aristas salientes.
Florida: Durazno (60).
Montevideo: Artigas (700), Canelones (30), Punta del Este (130).
Punta del Este: Rocha (90).
Rocha: Montevideo (270).

Parte 2. Algoritmo de Dijkstra

Partimos de Montevideo con distancia 0. Los vuelos directos son Artigas (700), Canelones (30) y Punta del Este (130).
1. Canelones (30): se actualizan las distancias a Artigas (530), Colonia (230), Durazno (200) y Punta del Este (120).
2. Punta del Este (120): se actualiza Rocha a 210 (120 + 90).
3. Durazno (200): no tiene vuelos salientes, por lo que no se actualiza nada.
4. Rocha (210): el vuelo a Montevideo da 480 (210 + 270), por lo que no mejora la distancia existente.
5. Colonia (230): el vuelo a Montevideo da 410 (230 + 180), por lo que tampoco mejora la distancia existente.
6. Artigas (530): el vuelo a Rocha da 930 (530 + 400), por lo que no mejora la distancia existente.
Florida queda con distancia infinita porque no existe un camino desde Montevideo hasta ese aeropuerto.

Resultado Final:

Destino	Distancia mínima	Trayecto
Canelones	30	Montevideo → Canelones
Punta del Este	120	Montevideo → Canelones → Punta del Este
Durazno	200	Montevideo → Canelones → Durazno
Rocha	210	Montevideo → Canelones → Punta del Este → Rocha
Colonia	230	Montevideo → Canelones → Colonia
Artigas	530	Montevideo → Canelones → Artigas
Florida	∞	No existe camino

Parte 3

3.2 Análisis de las interfaces

IGraph<V, D> es la interfaz base de cualquier grafo, dirigido o no. V es el tipo de los vértices y D el dato que guarda cada arista
(en este caso un WeightedEdge con la distancia). Define las operaciones comunes: agregar y remover vértices, agregar, eliminar y buscar
aristas, obtener los vértices, las aristas y las adyacencias de un vértice, y consultar si el grafo es conexo o tiene ciclos.
Los vértices se buscan con un Comparable<V> en lugar del vértice mismo, lo que permite buscar por un criterio (por ejemplo, el nombre
del aeropuerto sin importar mayúsculas). El método por defecto construirComparable arma ese criterio usando equals.
También trae métodos por defecto que se resuelven con los demás (agregarVertices, existeVertice, cantidadDeVertices, cantidadDeAristas),
así cada implementación solo tiene que programar las operaciones básicas.

IDirectedIGraph<V, D> extiende IGraph y agrega lo propio de un grafo dirigido, donde importa el sentido de las aristas:
successors (los vértices a los que se llega por aristas salientes) y predecessors (los vértices que tienen una arista hacia v).
A partir de ellos define por defecto gradoDeSalida y gradoDeEntrada. IUndirectedGraph cumple el mismo

papel para los grafos no
dirigidos, aunque no agrega operaciones porque ahí no hay sentido en las aristas.

IDirectedGraphAlgorithms separa los algoritmos de la estructura: el grafo solo guarda vértices y aristas, y los algoritmos lo reciben
por parámetro. Así se pueden aplicar a cualquier implementación de grafo. Define Dijkstra (caminos mínimos desde un origen), Floyd
(caminos mínimos entre todos los pares), Warshall (cerradura transitiva), la excentricidad y el centro del grafo, todos los caminos
entre dos vértices, los recorridos en profundidad y en amplitud y la clasificación topológica. Los que necesitan pesos exigen
D extends WeightedEdge. Los resultados se devuelven como interfaces (IDijkstraResult, IFloydWarshallResult, Path), que permiten
consultar el costo y recuperar el camino sin conocer cómo se guardaron internamente.

IUndirectedGraphAlgorithm agrupa los algoritmos de los grafos no dirigidos: Prim y Kruskal (árbol abarcador de costo mínimo),
searchMinEdge (la arista mínima entre dos conjuntos de vértices, auxiliar para Prim) y la búsqueda en amplitud. Se mantiene separada
de la de grafos dirigidos porque esos algoritmos solo tienen sentido sobre grafos no dirigidos.

3.3 Estructuras de datos utilizadas

Se utiliza un LinkedHashMap para almacenar cada vértice junto con su lista de aristas salientes.
Para guardar las aristas se utiliza ArrayList, mientras que para los conjuntos de vértices se utilizan LinkedHashSet y HashSet.
La implementación mediante listas de adyacencias consume O(V + E) de memoria, donde V es la cantidad de vértices y E la cantidad de aristas.
Esto resulta conveniente cuando el grafo tiene pocas aristas en relación con la cantidad de vértices, ya que una matriz de adyacencias
requiere $O(V^2)$ de memoria, independientemente de cuántas aristas existan realmente.
Las operaciones tienen distintos costos. Agregar un vértice cuesta O(1) en promedio, mientras que buscar un vértice cuesta O(V),
porque se recorren las claves del mapa. Agregar una arista cuesta O(V + g), ya que primero se buscan los vértices y después se comprueba si la arista ya existe, recorriendo las aristas salientes del origen. Eliminar una arista también cuesta O(V + g). Eliminar un vértice cuesta O(V + E), porque hay que buscarlo, eliminarlo y recorrer las aristas restantes para quitar las que llegan a él.
Obtener el conjunto de vértices cuesta O(1), porque se devuelve una vista de las claves del mapa.
Obtener todas las aristas cuesta O(V + E), porque se recorren las listas de adyacencias.
Buscar una arista cuesta O(V + g), ya que primero se busca el vértice de origen y después se recorren sus aristas salientes.
Obtener las adyacencias de un vértice cuesta O(V), porque primero hay que localizarlo.
Obtener los sucesores cuesta O(V + g), porque se busca el vértice y se recorren sus aristas salientes.
Obtener los predecesores cuesta O(V + E), porque se recorren las aristas de todos los vértices para encontrar cuáles llegan al vértice
indicado.
Vaciar el grafo tiene un costo O(1) para limpiar la estructura, sin contar el trabajo posterior del recolector de basura.
La detección de ciclos mediante búsqueda en profundidad tiene un costo promedio O(V + E), porque cada vértice y cada arista se procesan
como máximo una cantidad constante de veces.
También se podrían utilizar HashMap y HashSet en lugar de LinkedHashMap y LinkedHashSet. Estas estructuras ofrecen búsquedas promedio O(1),
pero no conservan el orden de inserción. LinkedHashMap y LinkedHashSet mantienen ese orden, aunque consumen algo más de memoria.
Si se necesita consultar aristas con frecuencia, se podría almacenar cada conjunto de aristas salientes en un mapa por destino.
De esta forma, se podría buscar una arista en tiempo promedio O(1) una vez localizado el vértice de origen, a cambio de consumir más memoria.
Por tanto, se considera adecuada la implementación mediante listas de adyacencias, ya que ofrece un

buen equilibrio entre el consumo de
memoria y el tiempo de ejecución.
La elección de las colecciones depende de si se prioriza conservar el orden de inserción, reducir el consumo de memoria o
acelerar las búsquedas de aristas.